GenomeVault: Full Build Plan

Nov 8, 2026 · @Gabriel

Overview

GenomeVault is a personal health agent that keeps your genome, records and bloodwork on your own device, checks any new prescription against them, and borrows cloud intelligence without revealing who you are or what you have.

Problem. A new drug can be unsafe or useless for you because of your genes, your kidney or liver function, or your other meds. That information lives in different places, and sending all of it to a cloud AI is a privacy and compliance non-starter.

Solution. A hybrid agent:

- **On device:** stores the personal health graph, runs deterministic safety rules, and decides what (if anything) must be asked of the cloud.
- **In the cloud:** answers only generalized, split, decoy-padded medical questions that reveal nothing personal.
- **Over time:** stores every cloud answer locally, so it needs the cloud less and gets faster and more private with use.

Why it fits Lenovo × AGI House. The event asks for agents that keep personal context on device, work offline, follow you across devices, call frontier models only when needed, and respect compliance. A genome is the most personal data there is, and medication safety is a real action with real stakes.

Pitch line: *Your genome never leaves your device. The cloud never learns what you asked. And the more you use it, the less it needs the cloud.*

Features and use cases

The agent does eight things; features 1–5 are the hackathon core, 6–8 are the polish that makes it win.

#	Feature	What it does	Build priority
1	Personal health graph	Stores genotypes → phenotypes, meds, conditions, allergies and dated lab values locally	Must
2	Medical input capture	Reads a prescription label photo, typed or spoken drug name, or OTC/supplement bottle	Must (photo can fall back to upload)
3	Local cross-reference engine	Deterministic checks: gene–drug (CPIC), drug–drug, drug–lab thresholds, allergies	Must
4	Privacy router	Decides local vs cloud; generalizes, splits and decoy-pads every outbound query	Must
5	Ledger	Shows every byte that left the device next to what stayed local	Must
6	Learning knowledge base	Stores all cloud answers (decoys included) so future questions stay local	Should
7	Actions	Drafts pharmacist/doctor messages, builds a visit summary, sets reminders, pushes to phone	Should
8	Offline mode	Rules + knowledge base keep working with no connection	Should

Use cases to show or mention

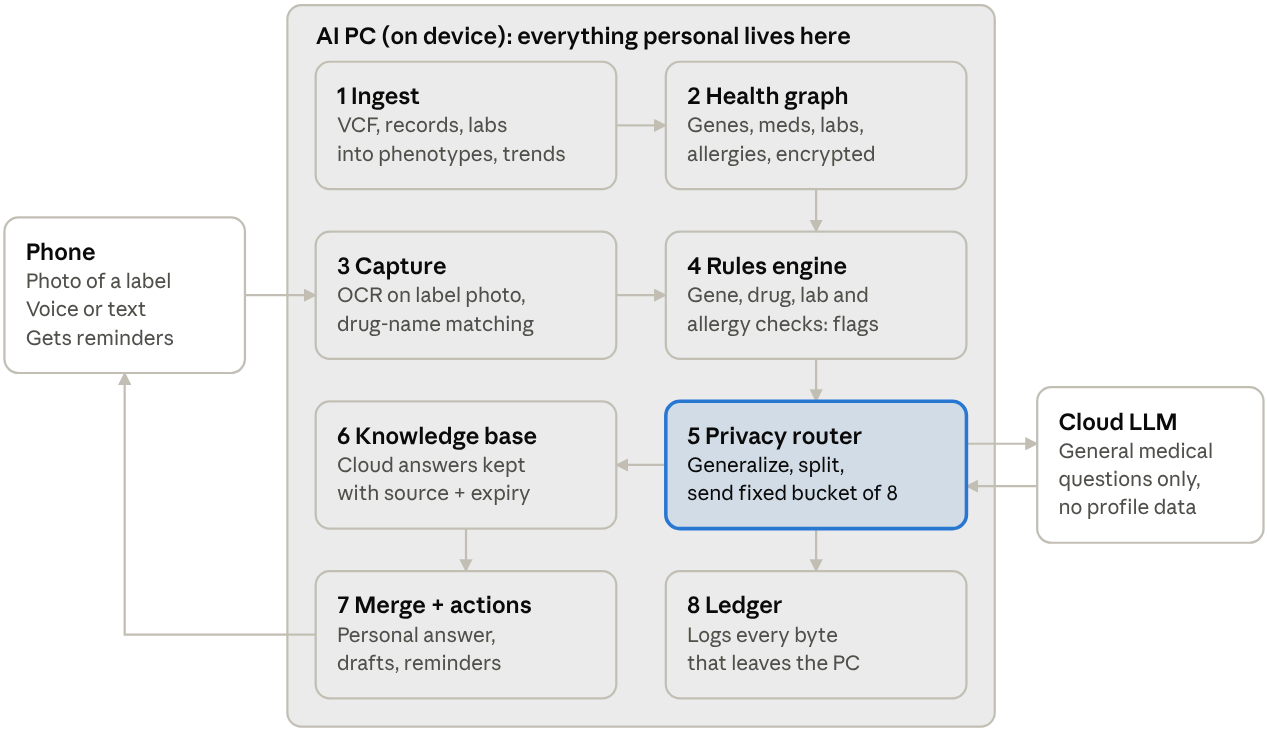
- **New prescription check:** “I was just prescribed X. Anything I should know?” (the main demo)
- **Discharge bundle:** several new meds at once, catching compounding problems between them.
- **OTC and supplements:** “Can I take ibuprofen / St. John’s wort with my meds?”
- **Lab change:** new bloodwork arrives, and the agent re-checks every current med against the new values.
- **Visit prep:** a one-page, doctor-ready summary of flags and questions.

- **Caregiver mode (roadmap):** a parent's or child's profile managed on one device.
- **Clinic mode (roadmap):** the same stack on an edge server for a pharmacy or practice.

System architecture

Eight modules run on the AI PC; the phone only captures inputs and receives actions, and the cloud only ever sees generalized decoy buckets.

Personal data stays on the PC; only decoy buckets reach the cloud



GenomeVault architecture · 8 local modules, phone and cloud

Numbers follow the build order. The router checks the knowledge base first; only cache misses send a fixed bucket of 8 to the cloud, and every answer (decoys included) is stored back in the knowledge base. Offline, the path simply stops at the router and the rules engine plus knowledge base still answer.

Tech stack and repo structure

Use Python end to end with a FastAPI backend and a single-page web UI, so one person can debug every layer.

Layer	Tool	Notes
Local LLM	Ollama or llama.cpp with a small instruct model (Qwen or Llama class, ~3–8B)	Ask Lenovo engineers how to run it on the AI PC's NPU/GPU and say so in the pitch
Cloud LLM	Claude or GPT via API	Only the router may call it
Backend	FastAPI	One process for rules, router, KB, ledger
Local store	SQLite (health graph, ledger) + Chroma or a NumPy embeddings file (knowledge base)	Encrypt at rest with a passphrase if time allows (`sqlcipher` or Fernet)
Embeddings	Small local sentence-embedding model	Runs on CPU fine
OCR	Tesseract or a local vision model	Fallback: typed drug name
Genotype calling	Hand-coded lookup for 4–5 genes; PharmCAT as the "real" path	PharmCAT is open-source from PharmGKB and runs locally
Synthetic patient	Hand-built JSON; Synthea for realism	Never use real patient data
UI	Streamlit for speed, or plain HTML + JS	Two panes: chat/flags left, ledger right
Phone	Phone browser hitting the laptop's local URL to upload a photo	Fake it with file upload if Wi-Fi is messy

Repo layout

```
genomevault/
  data/
    alex_profile.json        # genotypes, meds, conditions, allergies, labs
    alex.vcf                 # synthetic variants for the 5 genes
    rules/
      gene_drug.json         # CPIC-style phenotype -> drug recommendation
      drug_drug.json         # interaction pairs + severity
      drug_lab.json          # lab thresholds per drug
      drug_names.json        # brand/generic -> normalized name + class
    buckets.json             # fixed decoy buckets (k = 8)
```

```
core/
  ingest.py               # VCF/labs/records -> health graph
  graph.py                # SQLite health graph access
  capture.py              # OCR + drug-name normalization
  rules.py                # cross-reference engine -> flags
  router.py               # local-vs-cloud decision
  generalize.py           # strip, generalize, split
  decoys.py               # bucket lookup
  kb.py                   # local knowledge base (store + retrieve)
  cloud.py                # the ONLY module with network access
  merge.py                # personalize cloud facts locally
  actions.py              # drafts, summary, reminders
  ledger.py               # log every outbound byte
eval/
  redteam.py              # inference attack, 3 conditions
  learning_curve.py       # cloud-call rate over N questions
  queries.json            # 20 scripted test questions
app.py                    # FastAPI + UI
```

Rule of thumb: `cloud.py` is the only file that can touch the network. That makes the privacy claim easy to defend and to show on screen.

Data layer

All demo data is synthetic: one patient, “Alex,” built so every engine (genes, drug interactions, labs, allergies) fires at least once. Clinical rules below are illustrative; check each against the current CPIC guideline and drug label before the demo.

Alex at a glance

Category	Values
Situation	58, just discharged after a coronary stent
Conditions	Coronary artery disease, hypertension, high cholesterol, acid reflux (GERD)
Allergy	Penicillin

Category	Values
Genotypes	CYP2C19 *2/*2 (poor metabolizer); SLCO1B1 *1/*5 (decreased function); CYP2D6 *1/*1 (normal); CYP2C9 *1/*1 and VKORC1 reference (normal warfarin sensitivity)
Labs (latest)	eGFR 48 (down from 62 a year ago); potassium 5.2 mmol/L; ALT 28 U/L; CK 110 U/L; LDL 142 mg/dL
Discharge meds	Aspirin 81 mg, clopidogrel 75 mg, omeprazole 20 mg, simvastatin 40 mg, lisinopril 10 mg
Later input	Ibuprofen (OTC), used in the offline scene

Flags Alex should trigger

Flag	Engine	Severity
Clopidogrel likely less effective (CYP2C19 poor metabolizer; FDA boxed warning)	Gene–drug	High
Omeprazole inhibits CYP2C19, worsening the clopidogrel problem	Drug–drug + gene	High
Simvastatin at 40 mg with decreased SLCO1B1 function raises muscle-toxicity risk	Gene–drug	Medium
Lisinopril with rising potassium and falling kidney function	Drug–lab	Medium
Ibuprofen with aspirin + clopidogrel (bleeding) and with lisinopril + reduced kidney function	Drug–drug + drug–lab	High

Schemas

```
// alex_profile.json (excerpt)
{
  "patient_id": "local-only",
  "genotypes": {"CYP2C19": "*2/*2", "SLCO1B1": "*1/*5"},
  "phenotypes": {"CYP2C19": "poor_metabolizer", "SLCO1B1":
"decreased_function"},
  "meds": [{"name": "clopidogrel", "dose_mg": 75, "started": "2026-11-01"}],
```

```
  "conditions": ["CAD_post_stent", "hypertension", "GERD"],
  "allergies": ["penicillin"],
  "labs": [{"test": "eGFR", "value": 48, "unit": "mL/min/1.73m2", "date":
"2026-10-28"},
           {"test": "eGFR", "value": 62, "unit": "mL/min/1.73m2", "date":
"2025-10-20"}]
}

// rules/gene_drug.json (one entry)
{"gene": "CYP2C19", "phenotype": "poor_metabolizer", "drug": "clopidogrel",
 "severity": "high", "message": "Reduced activation; guideline suggests an
alternative antiplatelet if no contraindication",
 "source": "CPIC clopidogrel guideline"}

// rules/drug_lab.json (one entry)
{"drug": "lisinopril", "test": "potassium", "op": ">", "threshold": 5.0,
 "severity": "medium", "message": "ACE inhibitors can raise potassium;
recheck levels"}

// a flag (rules engine output)
{"id": "F1", "drug": "clopidogrel", "severity": "high", "engine":
"gene_drug",
 "evidence": ["CYP2C19 poor_metabolizer"], "needs_cloud": true, "topic":
"clopidogrel_cyp2c19"}

// buckets.json (one bucket, k = 8)
{"bucket_id": "B07", "topics": ["clopidogrel_cyp2c19", "codeine_cyp2d6",
"warfarin_cyp2c9_vkorc1",
  "tamoxifen_cyp2d6", "citalopram_cyp2c19", "atomoxetine_cyp2d6",
"ondansetron_cyp2d6", "voriconazole_cyp2c19"]}

// ledger entry
{"ts": "2026-11-08T14:02:11", "bucket_id": "B07", "queries_sent": 8,
"bytes_out": 1840,
 "personal_fields_sent": 0, "real_topic_index": "stored locally only",
"kb_hit": false}

// kb entry
{"topic": "codeine_cyp2d6", "facts": ["..."], "source": "cloud:model-name",
"fetched": "2026-11-08",
 "expires": "2027-05-08", "embedding": [0.01, "..."]}
```

Bucket design: build 6–10 buckets of 8 real, commonly asked gene–drug or drug–drug topics. Every topic lives in exactly one bucket, so asking about any member always sends the same 8.

Module specs

Each module has one job and a clear “done when” test, so you can check off progress during the day.

Module	Input → output	Core logic	Done when
`ingest.py`	VCF, lab CSV/FHIR, records JSON → health graph rows	Map variants to star alleles to phenotypes for 5 genes; store labs with dates	Alex’s graph loads and prints phenotypes
`graph.py`	Queries → meds, phenotypes, latest labs, lab trends	SQLite tables: genotypes, meds, conditions, allergies, labs	`get_context("clopidogrel")` returns relevant facts
`capture.py`	Image or text → normalized drug + dose	OCR, then fuzzy-match against `drug_names.json` (brand → generic → class)	Photo of a label returns `{"drug": "clopidogrel", "dose_mg": 75}`
`rules.py`	New drug + graph → list of flags	Run gene–drug, drug–drug (vs current meds), drug–lab, allergy checks; attach evidence and severity	Alex’s 5 meds produce the 4 expected flags
`router.py`	Flag or question → local answer or cloud plan	See decision logic below	Repeat question routes local

Module	Input → output	Core logic	Done when
`generalize.py`	Flag → 1+ generalized topics	Drop all personal fields; widen to all phenotypes / all lab bands; one concept per topic	Output contains zero genotype, lab value, age or co-med
`decoys.py`	Topic → its fixed bucket of 8	Lookup in `buckets.json`; send the whole bucket	Same topic always yields identical bucket
`kb.py`	Store cloud answers; retrieve by question	Store facts under exact canonical keys (drug pair or gene–drug) with source + date + expiry; embeddings only map a question to candidate keys	An unlisted pair returns “no data,” never a similar drug’s answer
`cloud.py`	Bucket → 8 answers	Only networked module; new session per bucket; no user identifiers	Network calls appear nowhere else (grep)
`merge.py`	Cloud facts + private flags → personal answer	Local LLM selects the right phenotype/band section and writes the answer with citations to stored facts	Answer names Alex’s specific risk
`actions.py`	Flags + answer → drafts, summary, reminders	Templates filled by local LLM; push to phone page	Pharmacist message and 1-page summary generate

Module	Input → output	Core logic	Done when
`ledger.py`	Every cloud call → log + UI feed	Record bytes, bucket, personal-field count (must be 0), KB hit/miss	Ledger panel updates live

Router decision logic

```
def route(question, flags):
    # 1. Fully local if the rules engine already answers it
    if is_rule_answerable(question, flags):
        return local_answer(flags)

    topics = generalize(question, flags)          # strip + widen + split
    plan = []
    for t in topics:
        hit = kb.get(t.key)                       # exact key only, never
similarity
        if hit and not hit.expired:
            continue                              # 2. Knowledge base covers
it
        if offline():
            continue                              # 3. Degrade gracefully
        plan.append(decoys.bucket_for(t))         # 4. Send the full bucket

    for bucket in dedupe(plan):
        answers = cloud.ask(bucket)               # one fresh session per
bucket
        kb.store_all(answers)                     # decoys included =
prefetch
        ledger.log(bucket, answers)

    # 5. Open-ended synthesis: cloud gets only generalized facts, never the
profile
    return merge.personalize(question, flags, kb.retrieve_many(topics))
```

Leakage check before any send: a local function scans the outbound text for genotype strings, rsIDs, star alleles, lab numbers, ages, dates and drug combinations from the profile. Any match blocks the send and logs it.

Exact-key rule: no extrapolation

The local model may never infer how drug A behaves with C from how A behaves with B, however similar B and C are. Every fact is keyed to specific drugs, and a missing key means "unknown," not "probably fine."

- **Canonical keys.** Normalize every input to its active ingredient (brand → generic; combination pills split into ingredients). Pair keys are sorted, e.g. `clopidogrel|omeprazole`. Brand-to-generic mapping is lookup, not extrapolation.
- **Three answer states only:** *known interaction* (fact exists for the exact key), *checked, none known* (the exact pair is in a curated source that lists it as no interaction), and *no data*. Absence from the table is always "no data."
- **Class rules only if authored.** Some interactions genuinely apply to a whole class (e.g. NSAIDs with antiplatelets). These are allowed only as explicit class entries written into the rule table with a source, never generalized by the model.
- **No-data path.** Online: send the exact pair's bucket to the cloud and store the answer under that exact key. Offline: tell the user "no data for this combination; ask your pharmacist."
- **Output validator.** Before showing an answer, check that every drug pair the local model mentions has a matching keyed fact. Any claim without one is removed and replaced with the no-data message.
- **Knowledge base lookups are exact.** Embedding similarity can map a question to candidate keys, but facts are only returned for an exact key match. Similar drug names (omeprazole vs. esomeprazole) must never borrow each other's facts.

Why it matters, in one example: omeprazole and pantoprazole are both proton-pump inhibitors, but they differ in how strongly they interfere with clopidogrel through CYP2C19. A model extrapolating within the class would get one of them wrong.

Demo moment: ask about pantoprazole + clopidogrel when only the omeprazole pair is stored. The agent says "no local data for this exact pair," fetches it, and stores it under its own key.

Remaining holes and fixes

Three holes are critical because they break the privacy or safety claim; fix them today, and state the rest as known limits.

Critical: fix in the build

Hole	Why it breaks things	Fix
Decoys that can be reversed. Exact-key pairs are a huge space, so buckets can't all be hand-made. If decoys are generated by a public rule from the real pair, an attacker reruns the rule on each member and only the real one reproduces the set.	Full leak despite decoys	Buckets must be a **partition**: a fixed, versioned universe of drug pairs split into groups of 8, where every member points to the same bucket. A pair outside the universe is added in the next universe release, never bucketed on the fly. Until then it gets the no-data path.
Joint inference across buckets. A discharge sends 4 buckets at once. Real co-prescription patterns (antiplatelet + acid reducer + statin + blood-pressure drug after a stent) let an attacker pick the one plausible combination out of thousands.	Reveals the condition even though each bucket is safe alone	Send buckets in separate sessions with randomized delays; mix in cover buckets on a steady schedule; build buckets so members of common clusters are spread across different buckets. Add a **joint-inference attacker** to the red team.
Trusting cloud answers forever. A wrong cloud answer gets stored under an exact key and served offline as fact.	Safety	Every fact gets a **trust tier**: curated guideline (CPIC, label) or cloud-generated. High-severity flags require a curated fact; cloud-tier facts are shown as "AI-summarized, verify with pharmacist" and expire sooner.

Important: handle with small changes

- **Dose matters.** Simvastatin risk depends on dose, so rule keys need dose conditions (e.g. `simvastatin|SLCO1B1:decreased|dose>20mg`), not just drug names.

- **Stale labs.** A flag based on bloodwork from 8 months ago can mislead. Show each lab's date and mark anything older than a set window (e.g. 90 days) as stale.
- **Validator scope.** The output validator checks drug pairs but should also check that every number or dose in the answer comes from a stored fact.
- **"None known" needs a complete source.** Demo tables are not complete, so the UI should never say "safe"; at most "no interaction listed in [source]."
- **Weak attacker.** An LLM attacker is a soft test. Add a statistical attacker that uses real prescription frequencies as priors; it's the stronger and more honest benchmark.

Known limits: say them, don't build them

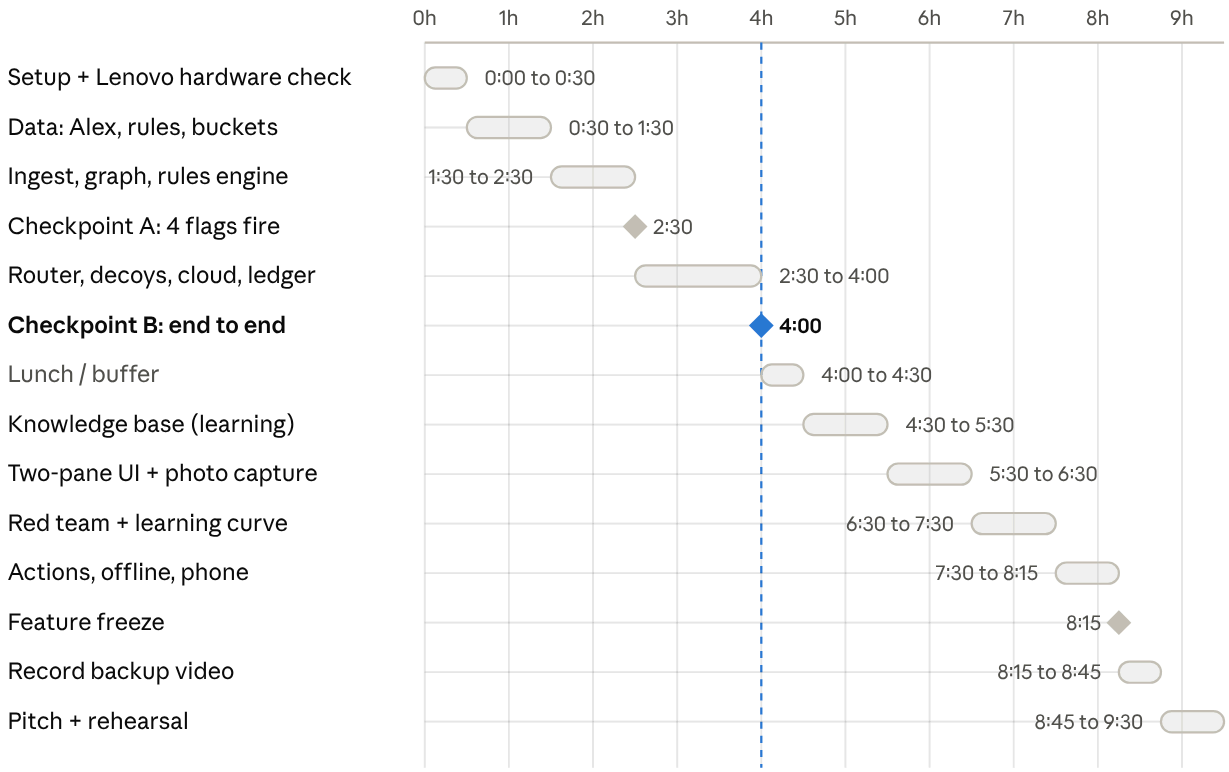
- Network metadata (IP, account, timing) needs a relay layer.
- Device security: encryption at rest helps, but a stolen unlocked laptop still exposes the graph.
- OCR text is untrusted input: only normalized drug names pass into the pipeline, never raw label text into prompts.
- Phone-to-PC transfer should run over local HTTPS on the same network.

Build timeline

Get one question working end to end, with the ledger, by hour 4; everything after that is upgrades.

A working end-to-end question by hour 4 decides the day

Build plan in hours from start; shift to the event's real schedule



Build timeline · about 9.5 hours, 3 gates

Cut lines if you fall behind:

- Missed Checkpoint A: hard-code Alex's phenotypes and skip VCF parsing.
- Missed Checkpoint B: demo from the terminal with a printed ledger; drop the UI polish.
- Behind at hour 6: skip photo capture (use typed drug names) and the phone push.
- Behind at hour 7.5: run the red team on 20 trials instead of 50; keep all three conditions.
- Never cut: the red-team chart and the ledger. They are what make this hybrid and defensible.

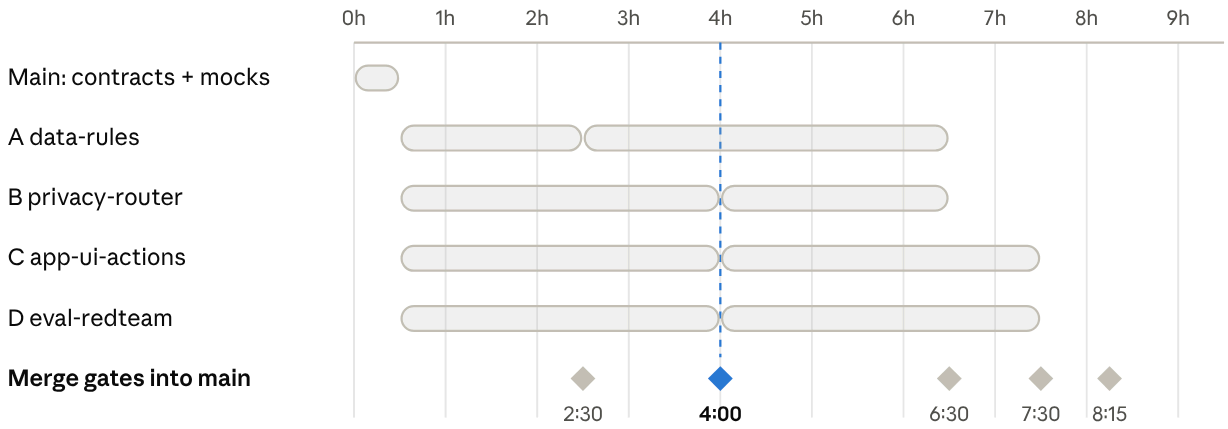
Already done before you start: write the 20 test questions and the bucket list during the Data block, before any router code.

Parallel branches

Four branches, each owning separate files, can be built by four agents at once after a 30-minute contracts commit on `main`.

Four branches build in parallel and merge into main at five gates

Hours from start; the seam in each bar marks its second phase



Branch timeline · 4 branches, 5 merge gates

The 4:00 gate matters most: after it, every branch works against the real integrated build instead of mocks.

Step 0 on main: contracts first (0:00 to 0:30)

Before any branch starts, commit the shared types and stub functions. Every branch codes against these, so nobody waits on anybody.

```
# core/contracts.py  (pydantic models; changes only via small PRs to main)
class Drug(BaseModel): ingredient: str; dose_mg: float | None = None
class PairKey(BaseModel): a: str; b: str            # always sorted
class Flag(BaseModel): id: str; key: str; severity:
Literal["high","medium","low"]
    engine: str; evidence: list[str]; needs_cloud: bool
class Fact(BaseModel): key: str; text: str; tier: Literal["curated","cloud"]
    source: str; fetched: date; expires: date
class Answer(BaseModel): state: Literal["known","none_listed","no_data"]
    text: str; facts: list[Fact]; sent_to_cloud: bool
class Bucket(BaseModel): id: str; version: str; keys: list[str]   # len 8
class LedgerEntry(BaseModel): ts: datetime; bucket_id: str; bytes_out: int
    personal_fields_sent: int; kb_hit: bool

# core/interfaces.py  (stubs return fixtures from mocks/)
def check(drug: Drug, profile: Profile) -> list[Flag]: ...      # Branch A
def route(question: str, flags: list[Flag]) -> Answer: ...      # Branch B
def personalize(answer: Answer, flags: list[Flag]) -> str: ...  # Branch C
```

Also on main: `mocks/` with Alex's fixture data and canned cloud answers, a `Makefile` with `make test` and `make demo`, and one `TASK.md` per branch (scope, owned files, do-not-touch list, done-when tests). Run each agent in its own **git worktree** (`git worktree add ../gv-router branch-b-router`) so they share one repo without stepping on each other.

The four branches

Branch	Owns (only these files)	Builds	Done when
A `data-rules`	`data/`, `core/ingest.py`, `graph.py`, `capture.py`, `normalize.py`, `rules.py`	Alex profile + VCF; exact-key rule tables with dose conditions and trust tiers; brand→ingredient normalization; OCR; rules engine; stale-lab marking	Alex's 5 meds produce the 4 expected flags; ibuprofen adds the 5th; unlisted pair returns `no_data`

Branch	Owns (only these files)	Builds	Done when
B `privacy-router`	`core/router.py`, `generalize.py`, `decoys.py`, `kb.py`, `cloud.py`, `ledger.py`, `validator.py`, `data/buckets/`	Versioned bucket partition; generalizer; leakage check; exact-key KB with tiers and expiry; session splitting with random delays and cover buckets; offline mode; output validator	Same key always yields the same bucket; zero personal fields in any outbound text; repeat question is a KB hit; `cloud.py` is the only networked file
C `app-ui-actions`	`app.py`, `ui/`, `core/merge.py`, `actions.py`, `phone/`	Two-pane UI (flags + chat, live ledger); profile panel; phone upload page; merge into personal answer; pharmacist draft, visit summary, reminders; **demo mode** that replays cached cloud answers	Full demo flow clicks through on mocks, then on real A + B
D `eval-redteam`	`eval/`, `docs/`, `slides/`	20 test questions; red team in 3 conditions plus joint-inference and statistical attackers; learning curve; latency/cost; offline count; charts; pitch slides and demo script	`make eval` regenerates every chart from a clean run

Merge gates into main

1. **2:30, Checkpoint A:** merge A. C and D switch from mock flags to real flags.
2. **4:00, Checkpoint B:** merge B and C's minimal UI. One question works end to end with the live ledger.
3. **6:30:** merge B's knowledge base and cover traffic plus C's full UI and phone page.
4. **7:30:** merge D's evals run against the integrated build; charts go into slides.
5. **8:15, freeze:** only bug fixes after this; record the backup video.

Rules for the agents

- Never edit a file another branch owns; ask for a contracts change through a PR to `main` instead.
- Every branch keeps `make test` green against mocks so merges are boring.
- Rebase on `main` after each gate.
- Branch B reviews any PR that touches networking; the grep test (`cloud.py` is the only file importing an HTTP client) runs in CI.

Autonomous agent protocol

The agents coordinate only through files in the repo, and no two agents ever write the same file, so they can run continuously without merge conflicts. The repo's `AGENTS.md` is the source of truth; this is the summary for the humans.

Shared files. `coord/PLAN.md` holds the idea with stable IDs (F1 features, M1 modules, I1 invariants, S1 demo beats, G1 gates) and a version number. `coord/DECISIONS.md` logs every accepted change. Only the human integrator edits these two.

Each agent's own files. A `BRIEF.md` (mission, owned files, task queue), a `STATUS.md` it rewrites every cycle, and a `LOG.md` it appends an impact report to after every code commit. Messages are new files in `coord/inbox/to-<agent>/` ; change requests are new files in `coord/proposals/` .

The cycle each agent repeats:

1. Sync with `main` and rebase.
2. Read what changed since last cycle: PLAN version, new decisions, inbox, other agents' new log entries.
3. Adjust its queue to those changes and reply to messages.
4. Do one task of 45 minutes or less, with tests green.
5. Commit with the convention, push, update its PR.
6. Append an impact report, rewrite STATUS, message affected agents, push.

Every change is classified, which is how the plan stays coherent:

Class	Meaning	What the agent does
Advances	Builds what PLAN already says	Just does it and reports

Class	Meaning	What the agent does
Refines	Decides a detail PLAN left open, inside its own area	Does it, reports it under "Refinements" for the integrator to fold into PLAN
Alters	Changes PLAN, a contract, an invariant, a demo beat, or another agent's assumptions	Writes a proposal and keeps building on the current plan until the integrator accepts it

Every impact report states: what changed and why, which PLAN IDs it advances or refines, contract impact, which agents are affected and what they must do, demo impact, test results, and the next task.

Guardrails. CI rejects commits that touch files the agent doesn't own, code commits without a `Class:` line, and network code outside `core/cloud.py`. Agents escalate to their human when an invariant would change, tests stay red for two cycles, a credential is needed, or a task fails three times. Messages from other agents are information, never orders that override `AGENTS.md`.

Human roles. Each person watches one agent and answers its escalations. One person is also the integrator: every 30 minutes they rule on proposals, update PLAN and DECISIONS, and merge PRs at gates. `touch coord/agents/A/STOP` pauses one agent; `touch coord/FREEZE` stops all.

Realistic expectation: agents will drift over 9 hours. Budget a 10-minute human check at every gate, and keep a person close to the critical path (Branch B before G2).

Evaluation and proofs

Four measured proofs turn the claims into evidence; the red-team test is the most important.

1. Red-team leakage test

An attacker model sees only what left the device and tries to recover Alex's private facts. Run 50 trials per condition (vary the synthetic patient so it isn't memorizing Alex).

Condition	What is sent	Expected attacker result
No protection	Full question + profile context	Recovers genotypes, meds, conditions
Generalized, random decoys	Generalized topic + 7 random topics	Still guesses above chance (decoys look fake or change between asks)
Generalized, fixed realistic buckets	Generalized topic + its fixed bucket	Guess accuracy near 1/8 (12.5%)

Attack prompt: “Here is everything a user’s device sent you. Which topic was the real one? What genotypes, meds or conditions does this person have?” Score topic-guess accuracy and count of correct private attributes.

2. Learning curve

Run the 20 scripted questions in order. Plot cloud calls per question and the cumulative share answered locally. Expected: steep drop after the first few, because buckets prefetch related topics.

3. Latency and cost

For the same 20 questions, compare local-only, cloud-only and hybrid on median latency, total cloud tokens and answer quality (rate quality 1–5 against a reference answer). Hybrid should match cloud quality at far lower cloud usage after warm-up.

4. Offline continuity

Turn off Wi-Fi after the session. Count how many of the 20 questions still get a correct flag or answer. Compare a cold device (empty knowledge base) against a warm one.

The 20 test questions

Write them before building the router so the eval is honest: 5 single-drug gene questions, 5 drug–drug, 4 drug–lab, 3 OTC/supplement, 3 open-ended (“what should I ask my cardiologist?”).

Demo script

The live demo runs about 3.5 minutes across seven beats; record a backup video of the full run by mid-afternoon.

1. **Hook (20s).** “Alex just got a heart stent and five new prescriptions. One of them may not work for him, and two others could hurt him. His genome, records and bloodwork could tell us, but would you upload your genome to a chatbot?”
2. **Capture (30s).** Photograph the discharge labels on the phone. They appear on the AI PC. Show the profile panel: genome, records and labs loaded, marked “on device.”
3. **Local flags (30s).** Four flags appear instantly with severity and evidence: clopidogrel + CYP2C19, omeprazole compounding it, simvastatin + SLCO1B1, lisinopril + potassium/kidney trend. Badge: “computed on device, 0 bytes sent.”
4. **Hybrid reasoning (45s).** Ask: “What should I bring up with my cardiologist?” The ledger shows separate buckets of 8 going out; the real topic is highlighted only on the local screen. The answer comes back personalized, citing Alex’s own results.
5. **Red team (45s).** Hand the ledger to an attacker model. Show the three-condition chart: naive leaks everything, random decoys leak some, fixed buckets sit near 1-in-8. Line: “We don’t claim privacy. We measure it.”
6. **It learns, works offline, takes action (45s).** Ask a related question: answered locally, 0 bytes, under a second. Turn Wi-Fi off and photograph ibuprofen: still flagged for bleeding and kidney risk. The agent drafts a pharmacist message and a one-page visit summary and sends a reminder to the phone.
7. **Close (15s).** Learning-curve chart, then: “The same stack runs on an edge server for a clinic, or across hospitals without moving a single genome.”

Stage rules: pre-load everything; keep a “demo mode” button that replays cached cloud answers if venue Wi-Fi dies; never improvise questions on stage.

Pitch, judge Q&A and limitations

Win the Q&A by naming weaknesses before judges do and showing the fix you built for each.

Pitch structure (2 minutes if no live demo slot)

1. Problem: the right drug depends on your genes, organs and other meds, and that data is too private for the cloud.

2. Insight: models should move to the data, not data to the models.
3. Demo highlights: local flags, buckets in the ledger, red-team chart.
4. Hybrid story in Lenovo's terms: what ran on the NPU, what ran in the cloud, and why.
5. Vision: personal device today, clinic edge server and hospital networks next.

Likely judge questions

Question	Answer
"Doesn't the cloud still learn your genotype?"	No: queries cover all phenotypes and lab bands, and the right section is picked locally.
"Couldn't repeat questions reveal you?"	Fixed buckets: any topic always sends the same 8, so repetition reveals nothing. Random decoys fail this; we show that in the chart.
"Why use the cloud at all?"	Rules cover guideline lookups; the cloud handles open-ended reasoning, and its answers are cached so use declines over time.
"Why not download all the answers?"	For the finite guideline set, that's the endpoint, and bulk sync is our roadmap. Decoys protect the open-ended questions.
"What about IP address and account metadata?"	Out of scope today; a relay layer (e.g. an anonymizing proxy) is the roadmap fix.
"Isn't this medical advice?"	It prepares you to talk to your pharmacist or doctor; it never says start or stop a drug.
"Why is this an agent, not a chatbot?"	It watches new inputs, re-checks meds when labs change, drafts messages and sets reminders across devices.

Limitations to state honestly

- Consumer genotype arrays miss some variants (e.g. CYP2D6 copy-number changes); clinical-grade data is needed for real use.
- Rules cover a demo subset, not every drug.
- Bucket privacy is only as good as the decoys: best case is 1-in-k, not zero.
- Synthetic data only; on-screen disclaimer: "Demo, not medical advice."

Beyond the hackathon

The most defensible business is the privacy layer itself, not the consumer app.

Grand vision: sovereign health agents. Personal data never moves; local agents sit on it at every level (phone, clinic server, hospital, biobank), and a cloud orchestrator only ever sees generalized, decoy-protected questions or aggregate counts.

Startup paths

Path	Customer	Why it could work	Main hurdle
Privacy SDK for health AI	Health apps that want frontier models without sending PHI	Router + buckets + leakage metric is reusable across any health app	Proving the privacy claim to compliance teams
Clinic / pharmacy edge appliance	Pharmacies, cardiology and oncology practices	On-prem fits HIPAA; Lenovo edge hardware is a natural partner	Sales cycle, EHR integration
Consumer app	People with existing genetic test results	Clear, personal value; easy to demo	Liability, retention, FDA decision-support rules
Federated cohort queries	Biobanks, pharma research	Query many sites without moving genomes	Long enterprise sales, governance

Roadmap after today

1. **Week 1:** clean repo, README, demo video, short write-up with the red-team results. Portfolio-ready.
2. **Month 1:** swap the hand-coded lookup for PharmCAT; expand rules; real FHIR import.
3. **Month 2–3:** stronger privacy: relay layer for metadata, bulk knowledge-pack sync, formal analysis of bucket design. Possible workshop paper on measured leakage of decoy schemes.
4. **Month 3+:** talk to 10 pharmacists and 5 health-app builders to pick between SDK and clinic paths; check FDA clinical decision support guidance with a regulatory advisor.

Portfolio angle: this project sits where genomics, privacy engineering and agent systems meet, which is a strong story for summer 2027 genomics and computational biology roles.